Relatório de Reengenharia

Grupo 6 — Refatoração do código legado com apoio de LLM

1. Ferramenta utilizada

Ollama, com o modelo `qwen2.5-coder:7b`.

2. Prompt utilizado

Texto enviado ao modelo, na íntegra:

CÓDIGO LEGADO

```
public class x_Mgr_F {
    public double calc(int tp, double[] vls, boolean st) {
        double r = 0;
        for (int i = 0; i < vls.length; i++) {
            r += vls[i];
        }
        if (tp == 1) {
            r = r - (r * 0.12);
            if (st) {
                r += 15.50;
            }
        } else if (tp == 2) {
            if (r > 500) {
                r = r - (r * 0.05);
            }
        }
        System.out.println("LOG: " + r);
        return r;
    }
}
```

DICIONÁRIO DE DADOS (obtido na engenharia reversa)

- `x_Mgr_F` = CalculadoraDeFaturamento
- `calc` = calcularValorFinal
- `tp` = TipoConsulta: 1 = CONVENIO, 2 = PARTICULAR, qualquer outro valor = sem regra (devolve o subtotal)
- `vls` = ProcedimentosRealizados (valor em reais de cada procedimento)
- `st` = cobrarTaxaAdministrativa (só tem efeito no convênio)
- `r` = subtotal / valor final
- `0.12` = percentual de desconto do convênio (12%)
- `15.50` = taxa administrativa do convênio (R$ 15,50)
- `500` = valor mínimo para desconto do particular (desconto só se subtotal for MAIOR que 500)
- `0.05` = percentual de desconto do particular (5%)

REGRAS DE NEGÓCIO

1. Somar todos os procedimentos para obter o subtotal.
2. Convênio: subtotal - (subtotal * 0.12); se cobrarTaxaAdministrativa for true, somar 15.50.
3. Particular: se subtotal > 500, aplicar subtotal - (subtotal * 0.05); senão, manter o subtotal.
4. Qualquer outro tipo: devolver o subtotal sem alteração.

O QUE VOCÊ DEVE FAZER

1. Usar nomes claros em português (Clean Code). Nada de nomes abreviados ou de uma letra.
2. Substituir os números mágicos por constantes com nome.

3. Aplicar o Princípio de Responsabilidade Única (SRP): uma classe apenas para somar os procedimentos e outras apenas para aplicar desconto/taxa.
4. Aplicar o padrão Strategy: uma interface (ex.: `RegraDeCobranca`) com uma classe por tipo de consulta (ex.: `RegraConvenio`, `RegraParticular`, `RegraSemDesconto`).
5. Aplicar o padrão Factory: uma classe que escolhe a regra correta a partir do tipo de consulta.
6. Criar um enum `TipoConsulta` que converta os códigos antigos (1 e 2).
7. Criar a classe `CalculadoraDeFaturamento`, que apenas orquestra: soma os procedimentos e depois aplica a regra.
8. NÃO usar `System.out.println` dentro das classes de regra de negócio.

RESTRIÇÕES OBRIGATÓRIAS

- O comportamento deve ser EXATAMENTE igual ao do legado (mesmos resultados para as mesmas entradas).
- Manter o tipo `double` e a mesma forma de cálculo (valor - valor * percentual). Não trocar por valor * 0.88.
- Código compatível com Java 8 ou superior, sem bibliotecas externas.

FORMATO DA RESPOSTA

- Um bloco de código separado para cada arquivo `.java`, com o nome do arquivo antes do bloco.
- Por último, uma classe `Main` que calcule: tipo 1 (convênio), procedimentos [200.0, 100.0], cobrarTaxaAdministrativa = true, e imprima o resultado. O valor esperado é 279.5.
- No final, explique em até 5 linhas por que o novo design é melhor que o legado.

3. Avaliação da resposta do LLM

Critérios verificados na resposta gerada pelo modelo:

Critério verificado	Resultado na resposta do LLM
Nomes claros (Clean Code), sem abreviações	**Atendido**
Números mágicos substituídos por constantes nomeadas	**Atendido**
SRP: soma separada da aplicação de desconto/taxa	**Atendido**
Padrões Strategy e Factory aplicados	**Atendido**
Sem `System.out.println` nas regras de negócio	**Atendido**
Particular: desconto só com subtotal **maior** que 500 (não maior ou igual)	**Preservado**
Cálculo na forma valor − valor × percentual (e não valor × 0,88)	**Preservado**
Tipo de consulta desconhecido devolve o subtotal sem alteração	**Preservado**
Taxa administrativa aplicada somente ao convênio	**Preservado**
Main prova que o resultado é igual ao do legado	**Não — ajustado (item 1 abaixo)**
Dependências da calculadora substituíveis (testabilidade / DIP)	**Não — ajustado (item 2)**
Validação de entrada nula	**Não — ajustado (item 3)**

4. Pontos ajustados

Critérios não atendidos pela resposta do LLM e que precisaram de ajuste manual:

- **Item 1 —** a classe `Main` deve provar que o resultado é igual ao do código legado.
- **Item 2 —** as dependências da `CalculadoraDeFaturamento` devem ser substituíveis (testabilidade / DIP).
- **Item 3 —** validação de entrada nula.